GPU-Accelerated TEBD for the 1D Transverse-Field Ising Model

Final Report, CME 213: Parallel Computing with CUDA, MPI and OpenMP

Hercy Shen, Chiling Han, Xin Wei — June 8, 2026

1. Project description

We simulate the **real-time quench dynamics** of the one-dimensional transverse-field Ising model (TFIM), a canonical quantum many-body system,

$$H = -J \sum_i \sigma_i^x \sigma_{i+1}^x \; - \; g \sum_i \sigma_i^z. \tag{1}$$

A chain of L spin-$\frac{1}{2}$ sites lives in a Hilbert space of dimension 2^L, so storing the wavefunction exactly is impossible beyond $L \approx 30$. The key physical fact that makes simulation tractable is that low-energy and short-time states of 1D local Hamiltonians have *bounded entanglement*, and can be compressed into a *Matrix Product State (MPS)*: the 2^L amplitudes are factored into L small rank-3 tensors connected by *bond* indices of dimension χ. The MPS stores only $O(L\chi^2 d)$ numbers ($d = 2$), where χ grows with the entanglement across each cut.

Computation. We evolve the state with *Time-Evolving Block Decimation (TEBD)*: the propagator e^{-iHt} is Suzuki–Trotter split into nearest-neighbour two-site gates $U_b = e^{-i\,\Delta t\, H_b}$, and each Trotter step applies $L - 1$ gates. Applying one gate is one small complex SVD that re-compresses the two-site tensor back to bond dimension χ; this SVD takes $> 99.6\%$ of GPU kernel time and is the target of all parallel work here.

- **Inputs:** chain length L; couplings (J, g); an initial product state; Trotter step dt; number of steps N; max bond dimension $\chi_{\max}$; Trotter order (1, 2, or 4).
- **Outputs:** time series of site magnetizations $\langle\sigma_i^z\rangle$, $\langle\sigma_i^x\rangle$, two-point correlations $\langle\sigma_i^x\sigma_j^x\rangle$, the bipartite von-Neumann entanglement entropy on every cut, the bond-dimension profile $\chi(t)$, and the accumulated SVD truncation error.
- **Assumptions/simplifications:** open boundary conditions, local dimension $d = 2$, nearest-neighbour bonds only.

Why this matters, and why parallelism. Entanglement grows under a quench, so $\chi(t)$ climbs over time (Fig. 1) until it saturates $\chi_{\max}$. Each bond update costs $O(\chi^3)$, so doubling the effective bond dimension is an $8\times$ hit and a (J, g) phase-diagram sweep grows expensive fast. A single GPU accelerates each evolution once χ is large, and multiple GPUs turn a minutes-per-point sweep into a study that finishes in wall-clock minutes.

2. Algorithms and state of the art

Core algorithm. We implement TEBD on a *right-canonical MPS* in Vidal form, standard for real-time evolution because it keeps the Schmidt spectrum Λ explicit on every bond and makes truncation locally optimal in the 2-norm. The two-site update is: (1) contract the gate with the two-site tensor Θ; (2) SVD the reshaped $(\chi_L d) \times (d\chi_R)$ matrix; (3) truncate to the largest $\chi_{\max}$ singular values and renormalize; (4) restore MPS form. The Trotter layer supports orders 1, 2 (symmetric), and 4 (Forest–Ruth), so the user can trade gate count for $O(dt^p)$ accuracy.

State of the art. Mature libraries (*TeNPy*, *ITensor*, *quimb*, *TensorNetwork*) are CPU/Python-centric and general; on GPUs, NVIDIA's *cuTensorNet* (cuQuantum) offers library-level batched contraction and SVD. The algorithmic variants relevant to our problem are the SVD engine, the time integrator (Trotter–Suzuki), and the parallel decomposition.

Our choice and how it compares. We deliberately do *not* try to beat cuTensorNet on raw throughput. Instead we build a fully instrumented TEBD in which every kernel is visible, so we can attribute time to gate contraction, SVD, truncation, and PCIe traffic without a library hiding the cost. For the SVD we chose *one-sided Jacobi*: at our matrix sizes ($2\chi \lesssim 256$) it converges in 5 - 10 sweeps and avoids the QR preprocessing that `gesvd` only amortizes on large matrices. The production path uses cuSOLVER's `gesvdj`; we additionally wrote a custom one-sided Jacobi CUDA kernel as the from-scratch deliverable and to quantify the gap to the standard library.

3. Parallelization strategy (CUDA + MPI)

3.1. The central difficulty and the decomposition we chose

A single TEBD chain has *limited spatial parallelism*. Within one Trotter sub-step the even bonds $\{0, 2, 4, \dots\}$ and odd bonds $\{1, 3, 5, \dots\}$ touch disjoint tensors and are independent ($\sim (L-1)/2$-way), but the even and odd sub-steps are *sequential*, and a spatial decomposition that splits the chain across ranks would require one MPI round-trip per sub-step to exchange the boundary Schmidt spectrum. At our target sizes ($L \leq 80$) the per-step compute is far too small to hide that latency.

We therefore use *task parallelism*: each MPI rank owns one GPU and runs complete, independent TEBD evolutions for a set of (J, g) points in the phase diagram, with zero in-loop communication, as production parameter studies do. We rejected two alternatives: (a) spatial chain decomposition, infeasible at small L due to per-sub-step synchronization; (b) replicated-parameter plus distributed chain, which needs per-bond MPI barriers.

3.2. Single-GPU bond-update pipeline

The bond update is implemented as a five-stage device pipeline. With $m = \chi_L d$, $n = d\chi_R$, $K = \min(m, n)$ and block shapes for $d = 2$:

#	Stage	Kernel / call	Notes
1	gate contraction	`gate_contract_kernel<2>`	gate in **shared memory**; `#pragma unroll`
2	row→col layout	`row_to_col_kernel`	**coalesced** reads for column-major SVD
3	SVD	`cusolverDnZgesvdj`	Jacobi, $K = \min(m, n)$, `lwork` re-queried per call
4	Vidal restore (L)	`vidal_restore_left_kernel<2>`	new$B_i[a, p, b] = U[ad{+}p, b]\, S[b]/\Lambda_a$
5	Vidal restore (R)	`vidal_restore_right_kernel<2>`	in-place conjugation of V

Every stage is bracketed by CUDA events so `bench_breakdown` can report per-stage time. The SVD is $> 99.6\%$ of kernel time (§5); all other kernels are $< 0.4\%$.

3.3. Persistent device-resident MPS and streams

- **Persistent device MPS (`GpuMps`).** All L B-tensors and Schmidt spectra stay device-resident for the whole evolution, dropping profiled H2D/D2H from tens of MB to ~ 0.2 MB; buffers are pre-allocated once (`GpuTebdWorkspace`), so there is zero `cudaMalloc` during evolution.
- **Streams within a sub-step.** `tebd_step_gpu_persistent` accepts n workspaces with independent CUDA streams and cuSOLVER handles. However, truncation requires a `cudaStreamSynchronize` mid-bond (to read singular values host-side), serializing bonds. Measured stream speedup is negligible (Table 1), a negative result the performance model predicts.

3.4. MPI decomposition and communication pattern

Each rank calls `cudaSetDevice(rank % n_gpus)`, builds its (J, g) points locally (no scatter), and runs independent evolutions; the TEBD loop contains no MPI calls. Only at the end does rank 0 collect results: an `MPI_Gather` of one `int` per rank (record counts), then an `MPI_Gatherv` of

variable-length result records (timing, $\chi_{\max}$, S_{mid}, and $2L + L^2$ observable values).

4. Performance model

We build a quantitative, falsifiable model from four pieces.

(a) Per-bond compute and the latency/compute crossover. The dominant cost is the SVD of a $2\chi_a \times 2\chi_a$ complex matrix. One-sided Jacobi does $O(\chi_a^3)$ flops per sweep over ~ 5–10 sweeps, plus a fixed overhead t_0 per bond:

$$T_{\text{step}}(L, \chi_a) \;\approx\; (L-1)\left(t_0 + c\,\chi_a^3\right). \tag{2}$$

Predictions: (i) when $c\chi_a^3 \ll t_0$ the code is **latency-bound**: T is linear in L and flat in $\chi_{\max}$; (ii) when $c\chi_a^3 \gg t_0$ the code is **compute-bound**: $T \propto \chi_a^3$.

(b) Roofline / arithmetic intensity. Each Jacobi sweep streams the $O(\chi_a^2)$-word matrix and does $O(\chi_a^3)$ flops, so arithmetic intensity $I = O(\chi_a)$ flops/byte *grows with* χ_a. Small matrices are below the ridge (latency-bound); large matrices climb past it (compute-bound).

(c) Communication model. The only message is the final `Gatherv` of $\lesssim 10$ KB. **Prediction:** communication fraction $< 10^{-4}$, invisible in a profiler.

(d) Parallel efficiency (task-parallel load balance). With P ranks and per-pair costs t_i:

$$E = \frac{\sum_i t_i}{P \max_r \sum_{i \in r} t_i}. \tag{3}$$

Pair cost rises steeply with J (more entanglement $\Rightarrow$ larger $\chi_a \Rightarrow O(\chi_a^3)$).

5. Benchmarking and instrumentation

Instrumentation. Per-stage GPU time measured with *CUDA events* (`bench_breakdown`); full-step wall time via host timers (`bench_gpu`); SVD-engine comparison via `bench_svd`; end-to-end timelines with *Nsight Systems* (`nsys profile --stats=true`), single-GPU and under `mpirun -n 4`.

Single-GPU full-step timing (`bench_gpu`, $L = 20$, 20 steps, `gpu-turing`):

$\chi_{\max}$	CPU (s)	M3 GPU (s)	Persistent (s)	Streamed (s)	CPU/Persistent
16	2.049	5.372	5.180	5.183	0.40
32	9.601	13.196	12.992	13.064	0.74
48	20.073	17.428	17.075	17.079	1.18
64	24.099	18.250	17.709	17.745	1.36
96	24.484	18.269	17.722	17.854	1.38
128	24.533	18.334	17.848	17.853	1.37

Table 1. GPU overtakes CPU at $\chi_{\max} \geq 48$; persistent MPS is 1–3% faster than M3 host-staging; four streams add nothing.

Per-stage kernel-time breakdown (`bench_breakdown` / Nsight `cuda_gpu_kern_sum`):

GPU kernel group	Share	Note
cuSOLVER `gesvdbj` + `svd_col/row_rotate`	96.4%	top-3 Jacobi SVD kernels
all remaining cuSOLVER helpers	3.2%	sort, scale, QR, etc.
`gate_contract_kernel` (ours)	0.09%	
`vidal_restore` left+right (ours)	0.13%	
`theta2` + `ss_inv` kernels (ours)	0.11%	persistent path
`row_to_col_kernel` (ours)	0.06%	
MPI communication	$< 0.01\%$	`Gatherv` of $\sim$10 KB at end

Table 2. cuSOLVER Jacobi is $> 99.6\%$ of GPU kernel time; all custom kernels together $< 0.4\%$.

Model vs. measurement. The crossover study (`bench_update`, Fig. 2) confirms prediction (a): at TFIM-realized $\chi_a \approx 41$ the GPU is latency-bound (flat across $\chi_{\max} = 64$–128 in Table 1), crossing the CPU near $\chi \approx 96$ and reaching $\sim 5\times$ at $\chi = 128$. Nsight confirms model (c): no MPI call appears in the timeline; every rank spends 96.2–96.5% of kernel time in the three Jacobi kernels.

6. Bottleneck analysis

The implementation has three regimes, each tied to hardware:

- **Single GPU, large χ: compute-bound on the SVD.** cuSOLVER's Jacobi kernels are $> 99.6\%$ of GPU kernel time (Table 2). A roofline argument corroborates this: the $2\chi_a \times 2\chi_a$ complex matrix fits in Turing's 4 MB L2 (256 KB at $\chi_a = 64$), so each element is loaded from HBM once per sweep, giving cache-resident intensity $I \approx 9n/16$ flops/byte ($n = 2\chi_a$). At $\chi_a = 64$, $I \approx 72$ flops/byte, roughly $95\times$ above the Turing ridge ($509\,\text{GFLOP/s} \div 672\,\text{GB/s} \approx 0.76$), placing the SVD deep in the compute-bound region; PCIe traffic after persistent MPS is only ≈ 0.24 MB H2D / 0.18 MB D2H.
- **Single GPU, small χ: latency-bound.** At TFIM-realized $\chi_a \approx 41$, every SVD matrix is small (82×82 complex), so the absolute flop count is too small to saturate the GPU. Per-bond time is set by kernel launch and cuSOLVER dispatch overhead t_0, and the GPU only wins once $\chi_{\max} \geq 48$ (Table 1). The custom gate and restore kernels ($O(\chi^2)$ work, $< 0.4\%$ of kernel time) are always launch-bound.
- **Multi-GPU: load imbalance, not communication.** Computation is $> 99\%$ of wall time; the final `Gatherv` is $< 0.01\%$ (Table 2). The distributed bottleneck is purely **load imbalance**: round-robin gives one rank the four hardest ($J = 2.0$) pairs, which run $2.2\times$ longer than the cheapest, and explains the $\sim 19\%$ efficiency gap at $P = 4$ (§8). The phase-diagram heatmap (Fig. 3) shows this directly: χ_a varies from 21 (cheap, low-J) to a saturated 64 (expensive, high-J), a $3\times$ spread in matrix size that becomes a $27\times$ spread in SVD flops.

A secondary bottleneck is the **mid-pipeline host sync** for truncation, which prevents stream overlap; removing it would require device-side top-k selection.

7. Algorithmic variants and their effect on performance

We explored four variants; each shifts the bottleneck differently.

1. **Host-staging (M3) → persistent device MPS.** Eliminating per-bond bulk H2D/D2H cuts profiled PCIe traffic from tens of MB to ~ 0.2 MB for a steady 1–3% full-step speedup (Table 1); the gain is small because the SVD already dominates.
2. **Single stream → 4 streams.** Negligible (Table 1, Fig. 4). The model predicted it; profiling showed why: host-side truncation serializes bonds.
3. **cuSOLVER `gesvdj` vs. custom one-sided Jacobi SVD.** Our hand-written Jacobi kernel is **1.2–2.2× slower** than cuSOLVER and agrees to $\lesssim 10^{-12}$ (Table 3, Fig. 5). Both paths plateau once $\chi_{\max}$ exceeds the realized $\chi_a \approx 41$–64 (cuSOLVER time changes $< 1\%$ from $\chi_{\max} = 64$ to 256), a direct consequence of the latency-bound regime: SVD matrix size is set by χ_a, not $\chi_{\max}$.
4. **Trotter order (1/2/4).** Higher order trades more gates per step for $O(dt^p)$ accuracy; order 4 (Forest–Ruth) reaches the ED-validation floor ($\sim 10^{-6}$, §9) at larger dt.

$\chi_{\max}$	cuSOLVER (s)	Custom Jacobi (s)	Ratio	max $\lvert\Delta\sigma\rvert$
16	7.110	8.184	0.869	4.74×10^{-14}
32	17.352	27.089	0.641	5.84×10^{-13}
48	22.688	46.056	0.493	9.96×10^{-13}
64	23.751	52.160	0.455	1.00×10^{-12}
128	23.754	52.208	0.455	1.00×10^{-12}
256	23.985	52.435	0.457	1.00×10^{-12}

Table 3. `bench_svd`*: cuSOLVER vs. custom Jacobi,* $L = 20$*, 20 steps. Both paths plateau above* $\chi_{\max} = 64$*; the wide-matrix adjoint fix reduced error from* $O(10^{-1})$ *to* $\lesssim 10^{-12}$*.*

8. Scalability analysis

We study scaling of the task-parallel sweep on `gpu-turing`, using total rank wall time $T(P) = \max_r \sum_{i \in r} t_i$.

Mode	Ranks	Pairs/rank	Time (s)	Speedup	Efficiency
strong	1	16	321.6	1.00	100%
strong	2	8	184.3	1.75	87%
strong	4	4	99.1	3.24	81%
weak	1	4	17.1	n/a	100%
weak	2	4	59.1	n/a	29%
weak	4	4	103.5	n/a	17%

Table 4. MPI strong and weak scaling. Strong speedup relative to $P = 1$ *wall time (321.6 s).*

Strong scaling. Speedup reaches 3.24× at $P = 4$ (81% efficiency, Fig. 6). The 19% loss is exactly the load imbalance the model predicted: round-robin hands rank 3 the four $J = 2.0$ pairs (highest entanglement), which run $\sim 2.2\times$ longer than rank 0's cheap $J = 0.25$ pairs. Communication is $< 0.01\%$.

Weak scaling. Efficiency falls sharply (29% at $P = 2$, 17% at $P = 4$), but this is *not* a communication or synchronization failure. The weak mode hands each added rank *higher-J* pairs, which are intrinsically more expensive (2–6× the cost of the $P = 1$ pairs). True constant-work weak scaling would require balancing by *estimated entanglement* rather than pair count.

Phase-diagram structure. Figure 3 shows the end-of-evolution mid-chain entropy and realized χ_a across the 4×4 grid: entropy peaks near the critical point ($J \approx g$) and saturates $\chi_{\max} = 64$ at most high-J points, while the paramagnetic corner reaches only $\chi_a \approx 21$–45. This 2–6× spread in χ_a sets the per-pair SVD cost and explains the load-imbalance gap.

Observables over the phase diagram. The sweep also collects the full set of GPU-computed observables ($\langle\sigma_i^z\rangle$, $\langle\sigma_i^x\rangle$, and $\langle\sigma_i^x\sigma_j^x\rangle$ for every pair; Fig. 7). The transverse magnetization $\langle\sigma_i^x\rangle$ is zero by parity: the all-$|0\rangle$ state has definite $\Pi = \bigotimes \sigma_i^z$, $\sigma_i^x \to -\sigma_i^x$ under Π, and H commutes with Π, pinning the expectation to zero. The ferromagnetic corner (large J, small g) shows the largest NN correlations and smallest $|\langle\sigma_i^z\rangle|$, as expected.

9. Correctness and verification

We verify correctness at four independent levels:

1. **Unit tests.** 54 Google Test cases across 6 binaries cover tensor ops, the hand-written GEMM/GEMV/SVD/`expm`/contraction/`kron`, MPS observables, the TFI model (Hermiticity, sum rules), and TEBD invariants (norm preservation, S^z conservation, entropy growth,

Trotter-order convergence). All pass at 10^{-12} (exact ops) / 10^{-10} (SVD-dependent).

2. **Analytical / exact-diagonalization oracle.** A full $2^L \times 2^L$ state-vector evolution validates TEBD on small systems. At $L = 8$, $dt = 0.05$, $t = 2$, order 4, $\chi_{\max} = 64$: max error 5.1×10^{-6} in $\langle \sigma^z \rangle$, 4.7×10^{-6} in $\langle \sigma^x \sigma^x \rangle$, 2.2×10^{-6} in entropy, and final fidelity loss 4.3×10^{-11}. The residual is dominated by the $O(dt^4)$ Trotter error; it sits at the Trotter floor from the first step onward (Fig. 8).
3. **CPU↔GPU agreement.** `validate_gpu` and `validate_persistent` run GPU and CPU reference side-by-side for 30 steps. Max discrepancies are 3.53×10^{-13} and 3.60×10^{-13}, at the floor where CPU Jacobi and cuSOLVER `gesvdj` agree on singular values. Custom Jacobi agrees to $\lesssim 10^{-12}$ (Table 3).
4. **Multi-rank consistency.** 1-, 2-, and 4-rank sweeps produce *identical* $(J, g) \to (\chi_{\max}, S_{\text{mid}})$ records, and 1-stream and 4-stream persistent configurations agree to every printed digit.

10. Discussion, limitations, and future work

What worked, and the largest payoffs. Task-parallel MPI with zero in-loop communication is the right fit at this scale and delivered 81% strong-scaling efficiency, limited only by load imbalance; on the single GPU, the persistent device MPS removed the M3 PCIe bottleneck cleanly.

What was not worth the complexity. *CUDA streams* within a sub-step gave negligible speedup because host-side truncation serializes bonds, and the *custom Jacobi SVD*, while valuable as a learning deliverable, stays 1.2–2.2× slower than cuSOLVER.

Limitations and future directions.

- **Load-balanced assignment.** Round-robin ignores the $\sim$ 2–6× runtime spread across (J, g); sorting pairs by estimated χ_a and bin-packing should lift strong-scaling efficiency toward $\gtrsim 95\%$ and remove the apparent weak-scaling collapse.
- **Device-side truncation.** A GPU top-k singular-value selection would remove the mid-pipeline host sync and let streams overlap bonds, the clearest path past the single-GPU SVD ceiling.
- **Spatial decomposition for $L \gg 100$.** A bond-cut decomposition would be needed when one evolution exceeds GPU memory or there are too few (J, g) points to fill the ranks, at the cost of one `MPI_Allgather` of the boundary Schmidt spectrum per sub-step; CUDA-aware MPI would then let the gather read device buffers directly, though the < 10 KB final transfer makes it immaterial today.

Connection to lecture. The project exercises the GPU memory hierarchy (shared memory for the gate, coalesced layout transforms, pre-allocation to avoid `cudaMalloc`), the roofline model (arithmetic intensity $O(\chi)$ separating latency-bound small SVDs from compute-bound large ones), MPI collectives, and Amdahl/load-balance analysis. The lesson: the useful parallelism here is *task-level*, the bottleneck is one dense kernel (the SVD), and the performance model predicted ahead of measurement both where the GPU wins and why streams and weak scaling disappoint.

Hardware/software. Benchmarks ran on the course partition `gpu-turing` (Quadro RTX 6000, `sm_75`, 24 GB, CUDA 12.3, OpenMPI), one GPU per rank; development also used a local RTX 5090 (`sm_120`) and a one-core `g++` baseline. All linear algebra is hand-written C++14/CUDA/MPI, with cuBLAS/cuSOLVER only for the production SVD.

References

1. G. Vidal, *Efficient classical simulation of slightly entangled quantum computations*, PRL 91, 147902 (2003); *Efficient simulation of one-dimensional quantum many-body systems*, PRL 93, 040502 (2004).

2. U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Ann. Phys. 326, 96 (2011).
3. R. P. Brent and F. T. Luk, *The solution of singular-value and symmetric eigenvalue problems on multiprocessor arrays*, SIAM J. Sci. Stat. Comput. (1985).
4. N. Hatano and M. Suzuki, *Finding exponential product formulas of higher orders* (Suzuki–Trotter decomposition).
5. NVIDIA cuSOLVER / cuQuantum (cuTensorNet) documentation.
6. TeNPy, ITensor, quimb (open-source tensor-network libraries).

A. Debugging notes

Three subtle bugs were caught with `compute-sanitizer` and auditing: (i) a gauge bug from cuSOLVER returning V (not V^H) in column-major, fixed by an explicit row→col transpose and conjugation in the right-restore kernel; (ii) a gate shared-memory load that zero-padded when a block had < 16 threads, fixed with a cooperative strided load; (iii) an out-of-bounds write from `gesvdj_bufferSize` being non-monotone in (m, n), fixed by re-querying `lwork` per call.

B. Supplementary figures

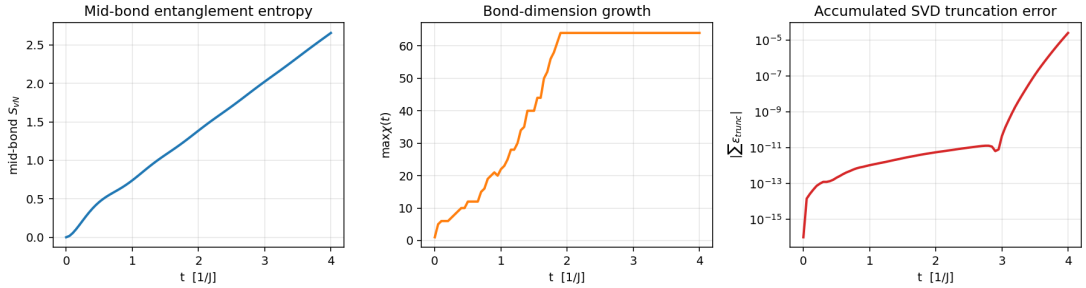

Figure 1: Reference $L = 20$ TFIM quench ($J = g = 1$, order 4, $\chi_{\max} = 64$). Entanglement growth drives bond-dimension growth, which sets the SVD sizes that dominate runtime.

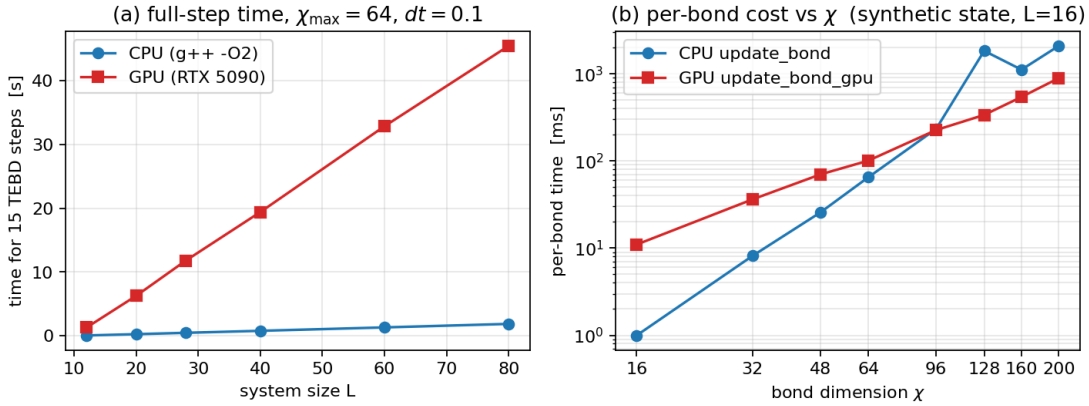

Figure 2: (a) Wall time vs. L at $\chi_{\max} = 64$: linear, ~ 1.3 ms/bond, latency-bound as predicted. (b) Per-call `update_bond` vs. χ: CPU/GPU crossover near $\chi \approx 96$–128.

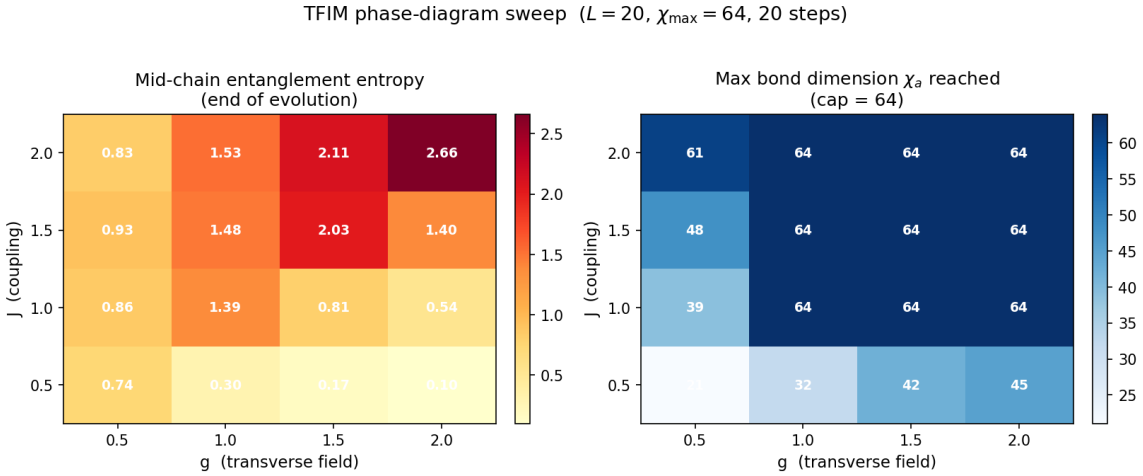

Figure 3: Left: final mid-chain entropy S_{mid}. Right: max realized bond dimension χ_a. High-J points saturate $\chi_{\max} = 64$ and dominate runtime; the paramagnetic corner (small J, large g) is cheap.

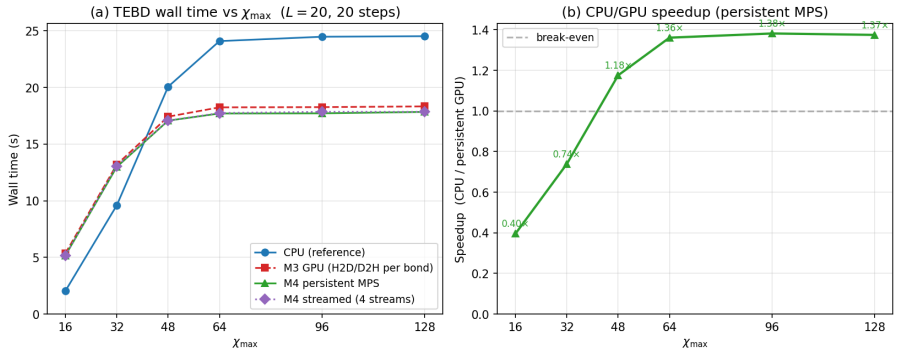

Figure 4: Single-GPU full-step timing vs. $\chi_{\max}$ (companion to Table 1).

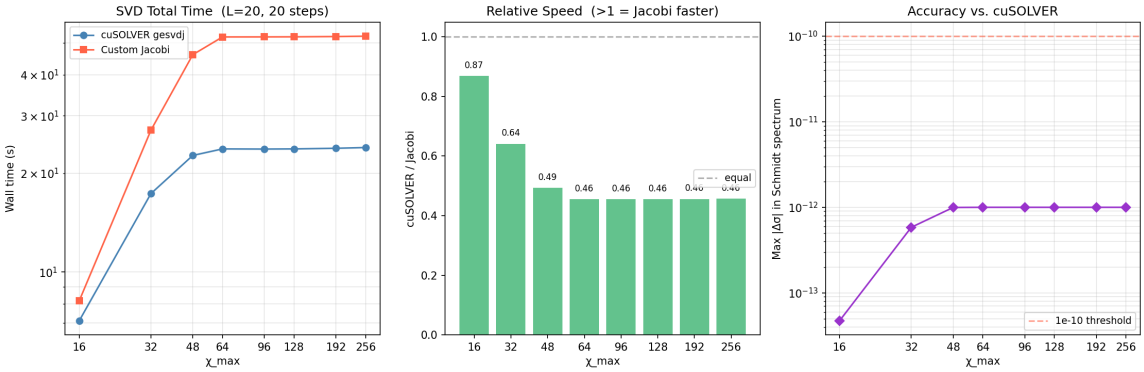

Figure 5: Custom Jacobi SVD vs. cuSOLVER `gesvdj`: time ratio (center) and Schmidt-spectrum error (right). Both plateau above $\chi_{\max} = 64$ (companion to Table 3).

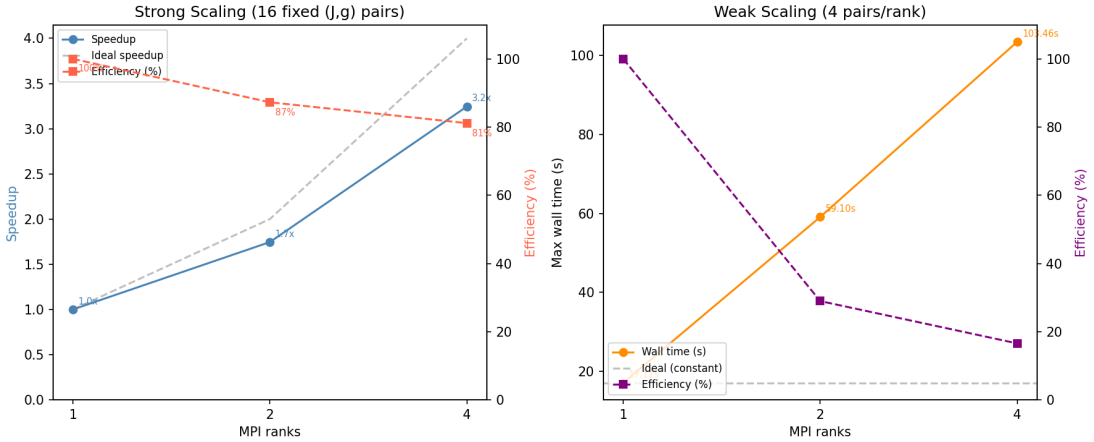

Figure 6: Strong-scaling speedup (left) and weak-scaling wall time (right) (companion to Table 4).

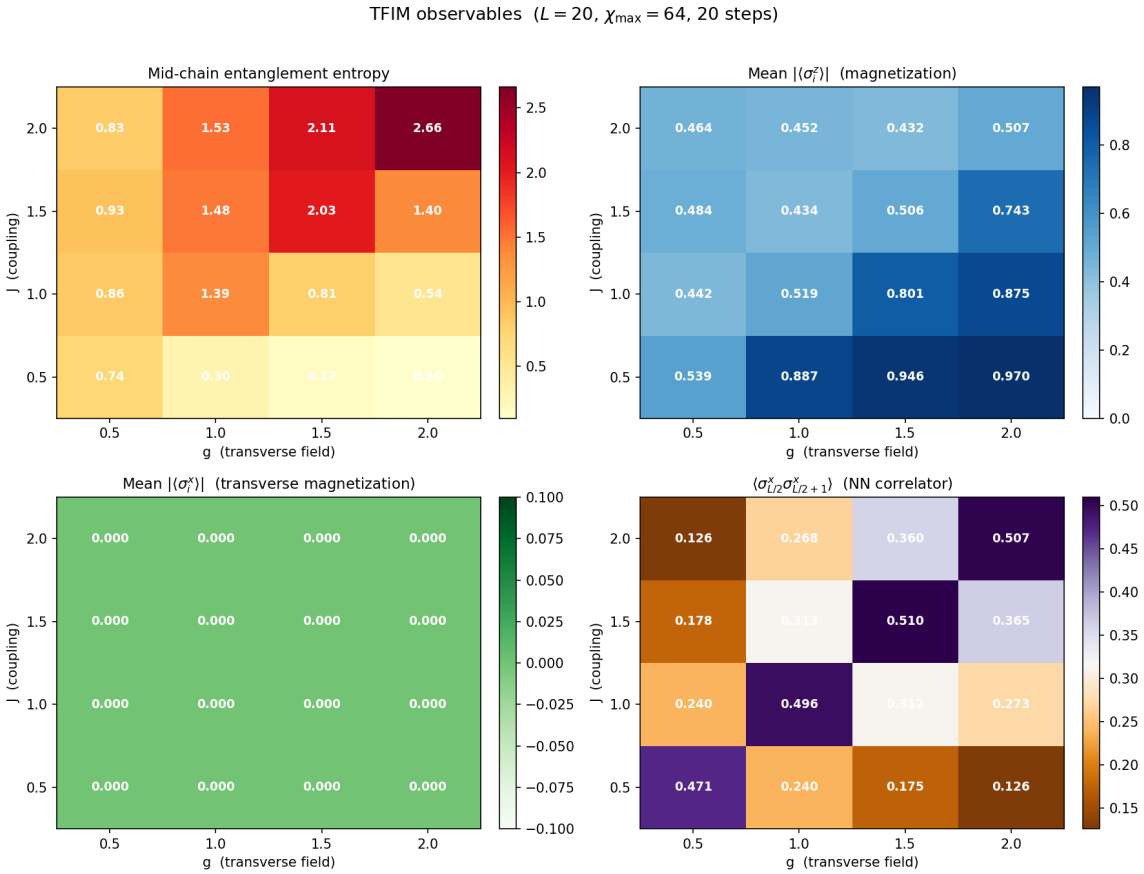

Figure 7: GPU-computed observables over the (J, g) phase diagram ($L = 20$, $\chi_{\max} = 64$, 20 Trotter steps at $dt = 0.1$). Top-right: mean $|\langle\sigma_i^z\rangle|$. Bottom-left: mean $|\langle\sigma_i^x\rangle| = 0$ (zero by parity). Bottom-right: nearest-neighbour x-correlator.

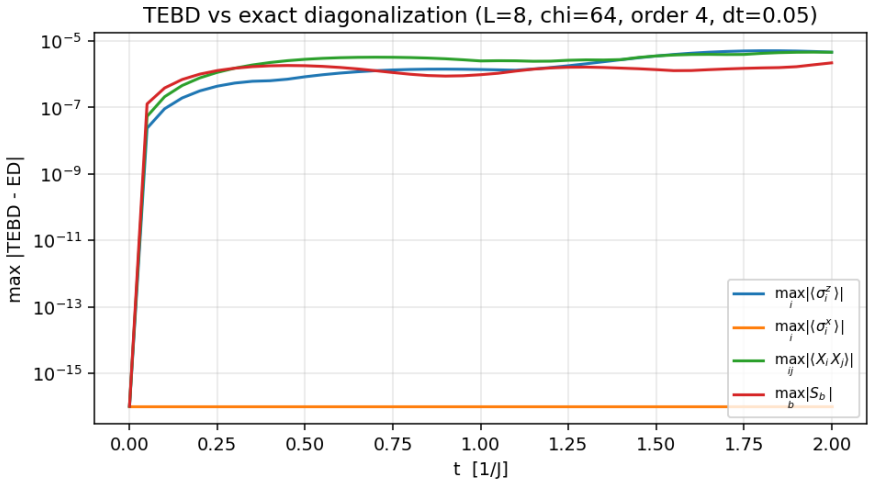

Figure 8: TEBD vs. exact diagonalization, per-step max error. Final fidelity loss 4.3×10^{-11}.